Data Science and Cloud Computing in Bioinformatics
Assignment – 7 HPC

Part A — Understanding the HPC Environment

```
(base) meron@DESKTOP-BUMHJQC:~$ ssh -X s_meron@197.156.76.122
s_meron@197.156.76.122's password:
(base) s_meron@betinserver:~$
```

```
(base) s_meron@betinserver:~$ pwd
/home/s_meron
```

```
(base) s_meron@betinserver:~$ hostname
betinserver
```

```
(base) s_meron@betinserver:~$ whoami
s_meron
```

```
(base) s_meron@betinserver:~$ df -h
Filesystem                         Size  Used Avail Use% Mounted on
tmpfs                               26G  6.8M   26G   1% /run
/dev/mapper/ubuntu--vg-ubuntu--lv   22T   21T  467G  98% /
tmpfs                              126G  1.1M  126G   1% /dev/shm
tmpfs                              5.0M     0  5.0M   0% /run/lock
/dev/sda2                          2.0G  213M  1.6G  12% /boot
/dev/sdb2                           51T  7.9T   41T  17% /media/Disk50TB
tmpfs                               26G   96K   26G   1% /run/user/124
tmpfs                               26G   84K   26G   1% /run/user/1001
tmpfs                               26G   84K   26G   1% /run/user/1010
tmpfs                               26G   84K   26G   1% /run/user/1054
tmpfs                               26G   84K   26G   1% /run/user/1064
```

```
(base) s_meron@betinserver:~$ du -sh
2.6G    .
```

```
(base) s_meron@betinserver:~$ squeue -u $USER
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODE
LIST(REASON)
```

Questions

A1. What is the purpose of a login node?

A login node is the entry point to an HPC cluster. Users connect to it to manage files, prepare jobs, load software modules, and submit jobs to the scheduler.

A2. Why should computationally intensive bioinformatics analyses not normally be run directly on the login node?

Login nodes are shared by many users and are intended for lightweight tasks. Running computationally intensive analyses on the login node can slow down the system and affect other users.

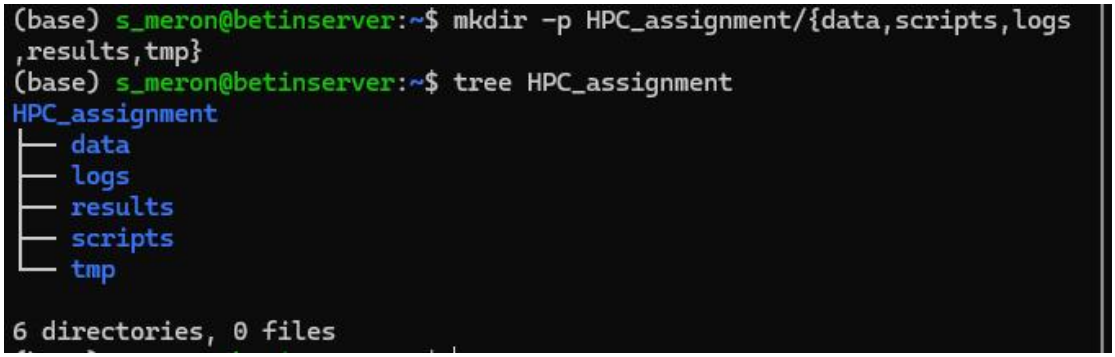
A3. What is the purpose of a compute node?

A compute node performs the actual computational work. Jobs submitted through SLURM are executed on compute nodes where CPU, memory, and other resources are allocated.

A4. What role does SLURM play between the user and the compute nodes?

SLURM is a workload manager that schedules jobs, allocates resources, manages queues, and dispatches jobs to appropriate compute nodes.

Part B — Create a Project Directory



Why separate data, scripts, logs, and results?

Separating directories improves reproducibility and organization. Raw data are kept separate from analysis scripts, logs record execution details and errors, and results are stored independently. This makes workflows easier to understand, reproduce, debug, and share.

Part C — Software Environment

Check for modules

```
(base) s_meron@betinserver:~$ module avail
module: command not found
```

Check whether the software is installed directly

```
(base) s_meron@betinserver:~$ module avail
module: command not found
(base) s_meron@betinserver:~$ which samtoolswhich samtools
/usr/bin/samtools
(base) s_meron@betinserver:~$ which bwawhich bwa
/usr/bin/bwa
(base) s_meron@betinserver:~$ which minimap2
(base) s_meron@betinserver:~$ which fastqc
/home/betin/anaconda3/bin/fastqc
(base) s_meron@betinserver:~$ which python
/home/s_meron/miniconda3/bin/python
(base) s_meron@betinserver:~$
```

If a package exists, get its version

```
(base) s_meron@betinserver:~$ samtools --version
samtools 1.19.2
Using htslib 1.20
Copyright (C) 2024 Genome Research Ltd.
```

```
(base) s_meron@betinserver:~$ which fastqc
/home/betin/anaconda3/bin/fastqc
(base) s_meron@betinserver:~$ which python
/home/s_meron/miniconda3/bin/python
(base) s_meron@betinserver:~$
```

```
(base) s_meron@betinserver:~$ minimap2 --version
Command 'minimap2' not found, but can be installed with:
apt install minimap2
Please ask your administrator.
```

The cluster does not appear to use the Environment Modules system. Software is available through direct installation on the system PATH rather than being loaded via modules.

C1. What is an environment module?

An environment module is a tool that dynamically modifies a user's shell environment, allowing different software packages and versions to be loaded or unloaded without affecting other users. Modules simplify software management on HPC systems.

C2. Why is recording the exact software version important?

Recording the exact software version is essential for reproducibility. Different versions may produce different results, contain different algorithms, or have different bugs and features. Specifying the version allows other researchers to reproduce the analysis accurately.

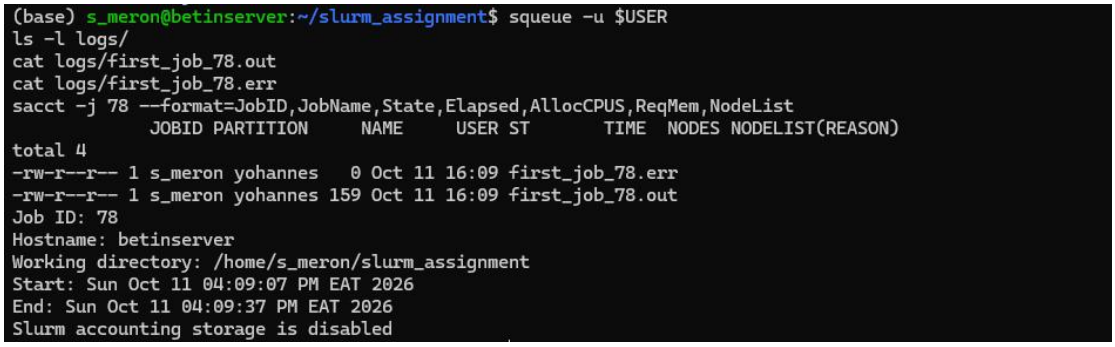
C3. Compare environment modules, Conda environments, and containers

Technology	Purpose	Advantages	Limitations
Environment Modules	Load software installed on the cluster	Lightweight and easy to switch versions	Depends on administrator-installed software
Conda Environments	Manage user-level packages and dependencies	User-controlled and reproducible	Can consume storage and may have dependency conflicts
Containers (Docker/Singularity)	Package software, dependencies, and OS libraries	Highly reproducible and portable	Larger and more complex than modules

Part D — Your First SLURM Batch Job

```
(base) s_meron@betinserver:~/slurm_assignment$ cat scripts/first_job.sh
chmod +x scripts/first_job.sh
#!/bin/bash
#SBATCH --job-name=first_job
#SBATCH --cpus-per-task=1
#SBATCH --mem=2G
#SBATCH --time=00:10:00
#SBATCH --output=logs/first_job_%j.out
#SBATCH --error=logs/first_job_%j.err

echo "Job ID: $SLURM_JOB_ID"
echo "Hostname: $(hostname)"
echo "Working directory: $(pwd)"
echo "Start: $(date)"
sleep 30
echo "End: $(date)"
```



--job-name
Sets a human-readable name for the job (here first_job). This name appears in squeue output, so you can pick your job out of a long list. It is only a label and does not change how the job runs.

--cpus-per-task
Sets the number of CPU cores allocated to each task. Here it is 1, meaning a single core, which is enough for a simple script that only prints text and sleeps. Programs that use multiple threads would request more cores.

--mem
Sets the amount of RAM requested for the job on each node. Here it is 2G, or 2 gigabytes. If the job tries to use more memory than requested, SLURM can kill it with an out-of-memory error. Requesting far more than needed wastes resources and can make the job wait longer in the queue.

--time
Sets the maximum wall-clock run time, in HH:MM:SS format. Here it is 00:10:00, or 10 minutes. If the job is still running when the limit is reached, SLURM terminates it. A realistic limit helps the scheduler fit your job in sooner.

--output
Sets the file where standard output (stdout) is written, which is the normal text printed by echo. In the script it is logs/first_job_%j.out, where %j is replaced by the job ID, so this run produced logs/first_job_78.out. Using %j gives each run its own file, so earlier logs are not overwritten.

--error
Sets the file where standard error (stderr) is written, which holds error messages and warnings. In the script it is logs/first_job_%j.err. Keeping it separate from the output log makes debugging easier, because errors are not mixed in with normal output. In this job the .err file was empty (0 bytes), which means no errors occurred.

$SLURM_JOB_ID
An environment variable that SLURM automatically sets inside a running job. It contains the unique numeric ID that SLURM assigned to the job. In your run it was 78, and it was printed on the first line of the log as Job ID: 78. The ID is used to identify

the job in commands like squeue, scancel and sacct, and it is what %j expands to in the log file names.

Part E — Monitoring a SLURM Job

```
(base) s_meron@betinserver:~/slurm_assignment$ sbatch scripts/first_job.sh && squeue -u $USER
Submitted batch job 79
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
                79     debug first_jo  s_meron PD       0:00      1 (None)
```

```
(base) s_meron@betinserver:~/slurm_assignment$ scontrol show job 79
JobId=79 JobName=first_job
   UserId=s_meron(1064) GroupId=yohannes(1007) MCS_label=N/A
   Priority=1 Nice=0 Account=(null) QOS=(null)
   JobState=COMPLETED Reason=None Dependency=(null)
   Requeue=1 Restarts=0 BatchFlag=1 Reboot=0 ExitCode=0:0
   RunTime=00:00:31 TimeLimit=00:10:00 TimeMin=N/A
   SubmitTime=2026-10-11T16:15:31 EligibleTime=2026-10-11T16:15:31
   AccrueTime=2026-10-11T16:15:31
   StartTime=2026-10-11T16:15:31 EndTime=2026-10-11T16:16:02 Deadline=N/A
   SuspendTime=None SecsPreSuspend=0 LastSchedEval=2026-10-11T16:15:31 Scheduler=Main
   Partition=debug AllocNode:Sid=localhost:4405
   ReqNodeList=(null) ExcNodeList=(null)
   NodeList=betinserver
   BatchHost=betinserver
   NumNodes=1 NumCPUs=2 NumTasks=1 CPUs/Task=1 ReqB:S:C:T=0:0:*:*
   ReqTRES=cpu=1,mem=2G,node=1,billing=1
   AllocTRES=cpu=2,node=1,billing=2
   Socks/Node=* NtasksPerN:B:S:C=0:0:*:* CoreSpec=*
   MinCPUsNode=1 MinMemoryNode=2G MinTmpDiskNode=0
   Features=(null) DelayBoot=00:00:00
   OverSubscribe=NO Contiguous=0 Licenses=(null) Network=(null)
   Command=/home/s_meron/slurm_assignment/scripts/first_job.sh
   WorkDir=/home/s_meron/slurm_assignment
   StdErr=/home/s_meron/slurm_assignment/logs/first_job_79.err
   StdIn=/dev/null
   StdOut=/home/s_meron/slurm_assignment/logs/first_job_79.out
   Power=
   TresPerTask=cpu:1
```

```
(base) s_meron@betinserver:~/slurm_assignment$ scontrol show job 79
JobId=79 JobName=first_job
   UserId=s_meron(1064) GroupId=yohannes(1007) MCS_label=N/A
   Priority=1 Nice=0 Account=(null) QOS=(null)
   JobState=COMPLETED Reason=None Dependency=(null)
   Requeue=1 Restarts=0 BatchFlag=1 Reboot=0 ExitCode=0:0
   RunTime=00:00:31 TimeLimit=00:10:00 TimeMin=N/A
   SubmitTime=2026-10-11T16:15:31 EligibleTime=2026-10-11T16:15:31
   AccrueTime=2026-10-11T16:15:31
   StartTime=2026-10-11T16:15:31 EndTime=2026-10-11T16:16:02 Deadline=N/A
   SuspendTime=None SecsPreSuspend=0 LastSchedEval=2026-10-11T16:15:31 Scheduler=Main
   Partition=debug AllocNode:Sid=localhost:4405
   ReqNodeList=(null) ExcNodeList=(null)
   NodeList=betinserver
   BatchHost=betinserver
   NumNodes=1 NumCPUs=2 NumTasks=1 CPUs/Task=1 ReqB:S:C:T=0:0:*:*
   ReqTRES=cpu=1,mem=2G,node=1,billing=1
   AllocTRES=cpu=2,node=1,billing=2
   Socks/Node=* NtasksPerN:B:S:C=0:0:*:* CoreSpec=*
   MinCPUsNode=1 MinMemoryNode=2G MinTmpDiskNode=0
   Features=(null) DelayBoot=00:00:00
   OverSubscribe=NO Contiguous=0 Licenses=(null) Network=(null)
   Command=/home/s_meron/slurm_assignment/scripts/first_job.sh
   WorkDir=/home/s_meron/slurm_assignment
   StdErr=/home/s_meron/slurm_assignment/logs/first_job_79.err
   StdIn=/dev/null
   StdOut=/home/s_meron/slurm_assignment/logs/first_job_79.out
   Power=
   TresPerTask=cpu:1
```

PENDING (PD): The job has been accepted and is waiting in the queue. It has not started yet, usually because resources (CPUs, memory) are busy or higher-priority jobs are ahead of it. The REASON column in squeue tells you why, for example Resources or Priority.

RUNNING (R): The job has been given resources and is actively executing on a compute node. Its commands, such as sleep 30, are being run.

COMPLETED (CD): The job finished and every command ended successfully, with exit code 0. This is the normal, successful end state, and it is what job 78 reached.

FAILED (F): The job ended with a non-zero exit code, meaning the script or program hit an error (for example a command not found or a program crash). The exit code shows what went wrong.

CANCELLED (CA): The job was stopped before finishing, either by the user (for example with scancel) or by an administrator. A job can be cancelled while pending or while running.

TIMEOUT (TO): The job ran longer than its --time limit, so SLURM terminated it. For example, if sleep 30 had been replaced by a task needing 20 minutes while --time=00:10:00 was set, the job would end as TIMEOUT.

OUT_OF_MEMORY (OOM): The job used more RAM than it requested with --mem, so SLURM killed it. The fix is to request more memory, or to reduce the program's memory use.

Wall time vs CPU time

Wall time (also called elapsed time) is the real time that passes on the clock from the moment the job starts until it ends, as if you were timing it with a stopwatch. SLURM's --time limit and the Elapsed field in sacct both measure wall time. Job 78 had a wall time of 30 seconds.

CPU time is the total time the CPU cores actually spent doing computation for the job, added up across all cores. It is roughly wall time multiplied by the number of cores in use, and it excludes time spent idle or waiting.

The two can differ a lot:

- A job that uses 4 cores fully for 10 minutes of wall time has about 40 minutes of CPU time.
- A job that just waits has low CPU time compared to its wall time. Our sleep 30 job has about 30 seconds of wall time but close to **0 seconds of CPU time**, because sleeping uses almost no CPU.

Wall time determines how long you wait and what you must set in --time. CPU time shows how much computing was actually done, and is often what clusters use for usage accounting and fair-share.

Part F — Multithreaded Bioinformatics Job

```
(base) s_meron@betinserver:~$ cd ~/slurm_assignment
nproc
sinfo -o "%n %c %m"
32
HOSTNAMES CPUS MEMORY
betinserver 2 4096
```

Submit and monitor

```
(base) s_meron@betinserver:~/slurm_assignment$ sbatch scripts/multithread.sh && squeue -u $USER
Submitted batch job 82
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
                82     debug multithr  s_meron PD       0:00      1 (PartitionConfig)
```

```
(base) s_meron@betinserver:~/slurm_assignment$ sbatch scripts/multithread.sh
squeue -u $USER
Submitted batch job 85
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
                85     debug multithr  s_meron PD       0:00      1 (PartitionConfig)
                84     debug multithr  s_meron PD       0:00      1 (PartitionConfig)
                83     debug multithr  s_meron PD       0:00      1 (PartitionConfig)
                82     debug multithr  s_meron PD       0:00      1 (PartitionConfig)
```

F1. What happens if you request 32 CPUs but the program uses only 1 thread?

Only one CPU does any work, and the other 31 sit idle for the entire job. The program runs no faster than it would with a single CPU, but SLURM has reserved all 32 cores for you, so nobody else can use them. The consequences are:

- **Wasted resources:** the idle cores are blocked from other users, so the cluster is underused.
- **Longer queue time:** a 32-CPU request is harder for the scheduler to fit, so your job may wait much longer before it starts.
- **Lower fair-share priority:** many clusters charge usage by the CPUs *allocated*, not the CPUs *used*, so you spend your allocation for no benefit and your future jobs may get lower priority.

F2. What could happen if SLURM allocates 4 CPUs but your program launches 32 threads?

The 32 threads must share 4 cores, which is called oversubscription. Possible results:

- **Slowdown:** the operating system constantly switches threads on and off the cores (context switching), which adds overhead. The job can end up slower than with 4 threads.
- **No speedup:** only 4 threads can run at the same time, so the other 28 add nothing.
- **Memory problems:** each thread uses some memory, so 32 threads may exceed the --mem limit and the job can be killed (OUT_OF_MEMORY).
- **Interference with other users:** on clusters that enforce CPU limits (cgroups), the job is confined to its 4 cores. On clusters that don't, it may spill onto cores allocated to other users and slow their jobs, which administrators treat as a misuse of the cluster.

F3. Why is program --threads "$SLURM_CPUS_PER_TASK" **preferred over hardcoding** program --threads 32**?**

- **It always matches the allocation:** the thread count equals what SLURM actually granted, which avoids both the idle CPUs of F1 and the oversubscription of F2.
- **It needs only one change:** to use 16 CPUs instead of 8, you edit just the #SBATCH --cpus-per-task line. With a hardcoded number you must update two places, and forgetting one creates a mismatch.
- **It is portable:** the same script works on any node or cluster, regardless of size, without rewriting.
- **It is safer and reproducible:** other people can reuse the script, and the job cannot accidentally use more than its fair share of the machine.

Part G — SLURM Job Array

```
(base) s_meron@betinserver:~/slurm_assignment$ cd ~/slurm_assignment
for i in $(seq -w 1 10); do echo "sample$i"; done > samples.txt
cat samples.txt
wc -l samples.txt
sample01
sample02
sample03
sample04
sample05
sample06
sample07
sample08
sample09
sample10
10 samples.txt
```

```
(base) s_meron@betinserver:~/slurm_assignment$ sbatch scripts/process_samples.sh && squeue -u $USER
Submitted batch job 87
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
         87_[1-10]     debug process_  s_meron PD       0:00      1 (None)
                85     debug multithr  s_meron PD       0:00      1 (PartitionConfig)
                84     debug multithr  s_meron PD       0:00      1 (PartitionConfig)
                83     debug multithr  s_meron PD       0:00      1 (PartitionConfig)
                82     debug multithr  s_meron PD       0:00      1 (PartitionConfig)
```

```
(base) s_meron@betinserver:~/slurm_assignment$ ls results/
ls results/ | wc -l
cat results/*_result.txt
sample01_result.txt  sample02_result.txt
2
sample01 processed successfully
sample02 processed successfully
```

G1. What is a SLURM job array?

A job array is a way to submit many similar jobs with a single sbatch command. SLURM creates one task per index (here --array=1-10 creates tasks 1 to 10). All tasks run the same script, but each gets a different $SLURM_ARRAY_TASK_ID, which the script uses to pick its own input, such as line N of samples.txt. The tasks are scheduled independently, so they can run in parallel as resources become free. All tasks share one array job ID ($SLURM_ARRAY_JOB_ID), and each can be tracked as <arrayID>_<taskID>.

G2. Why are arrays useful for FASTQ processing?

Sequencing projects usually have many samples, each with its own FASTQ files, and each sample needs the same steps (QC, trimming, alignment) independently of the others. This is an ideal fit for arrays:

- **One script, many samples:** you write and test the pipeline once instead of creating ten or a hundred separate scripts.
- **Parallelism:** samples are processed at the same time when resources allow, which shortens total time.
- **Easy to manage:** one sbatch submits everything, scancel cancels the whole set, and a failed sample can be rerun alone (for example --array=7) without redoing the others.
- **Organised output:** each task writes its own logs and result files, named by sample.
- **Fair to others:** you can limit how many tasks run at once (for example --array=1-10%2), so you don't flood the cluster.

G3. If each array task requests 8 CPUs and 10 tasks run simultaneously, what is the maximum CPU allocation?

Total CPUs = CPUs per task × number of simultaneous tasks
= 8 × 10
= 80 CPUs

The cluster would have to reserve 80 CPU cores at the same moment. If that much isn't free, some tasks stay pending until resources are released. You can cap this with a throttle such as #SBATCH --array=1-10%2, which would limit the allocation to 8 × 2 = 16 CPUs.

G4. If each task requires 16 GB RAM, what is the combined memory allocation for 10 simultaneous tasks?

Try = memory per task × number of simultaneous tasks
= 16 GB × 10
= 160 GB

This is the total memory reserved across the nodes. It does not have to sit on one node, because each task is placed on whichever node can fit it. Together with G3, 10 simultaneous tasks of 8 CPUs and 16 GB would need **80 CPUs and 160 GB RAM**. This is why array jobs on shared clusters are often throttled with %N.

Part H — Controlling Array Concurrency

CPU

Maximum CPUs = CPUs per task × concurrent tasks
= 8 × 20
= 160 CPUs

RAM

Maximum RAM = memory per task × concurrent tasks
= 24 GB × 20
= 480 GB

Why limiting concurrency is beneficial

- **Fair sharing:** a cluster is shared by many users. Without a limit, one array could occupy most of the cores and memory, leaving everyone else waiting.
- **Protects shared storage and network:** 1,000 tasks reading and writing FASTQ files at once can overload the file system, slowing every user's jobs, including yours. Fewer concurrent tasks means steadier I/O.

- **Avoids hitting limits:** clusters often cap the CPUs, memory or jobs per user. Exceeding the cap leaves tasks stuck pending, whereas a throttle keeps you within a predictable footprint.
- **Predictable, controlled resource use:** you know the most you will ever hold (160 CPUs, 480 GB), which makes planning and quota management easier.
- **Easier debugging and recovery:** if a bug or bad input causes failures, only about 20 tasks are affected at a time, not all 1,000, and you can cancel and fix without wasting huge amounts of compute.
- **Good cluster etiquette:** administrators may warn or restrict users who flood the scheduler, while a throttled array avoids this.

Part I — Job Dependencies

```
(base) s_meron@betinserver:~/slurm_assignment$ ALIGN_ID=$(sbatch --parsable scripts/alignment.sh)
echo "Alignment job ID: $ALIGN_ID"
VAR_ID=$(sbatch --parsable --dependency=afterok:$ALIGN_ID scripts/variant_calling.sh)
echo "Variant calling job ID: $VAR_ID"
squeue -u $USER
Alignment job ID: 97
Variant calling job ID: 98
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
                98     debug variant_  s_meron PD       0:00      1 (Dependency)
                85     debug multithr  s_meron PD       0:00      1 (PartitionConfig)
                84     debug multithr  s_meron PD       0:00      1 (PartitionConfig)
                83     debug multithr  s_meron PD       0:00      1 (PartitionConfig)
                82     debug multithr  s_meron PD       0:00      1 (PartitionConfig)
                97     debug alignmen  s_meron  R       0:01      1 betinserver
```

```
(base) s_meron@betinserver:~/slurm_assignment$ squeue -u $USER
cat logs/alignment_$ALIGN_ID.out
cat logs/variant_calling_$VAR_ID.out
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
                85     debug multithr  s_meron PD       0:00      1 (PartitionConfig)
                84     debug multithr  s_meron PD       0:00      1 (PartitionConfig)
                83     debug multithr  s_meron PD       0:00      1 (PartitionConfig)
                82     debug multithr  s_meron PD       0:00      1 (PartitionConfig)
Alignment job 97 started: Sun Oct 11 07:03:03 PM EAT 2026
Alignment finished: Sun Oct 11 07:03:33 PM EAT 2026
Variant calling job 98 started: Sun Oct 11 07:03:34 PM EAT 2026
Variant calling finished: Sun Oct 11 07:03:39 PM EAT 2026
```

Why are dependencies preferable to manually checking whether every preceding job has finished?

- **It is automatic:** SLURM starts the next job the moment the condition is met, with no delay. Manual checking means you must watch squeue repeatedly, possibly at night or over weekends, and the pipeline sits idle until you notice.
- **It is reliable:** the scheduler tracks the exit status for you. afterok runs the next step only if the previous one truly succeeded, so variant calling cannot start on a missing, partial or corrupted BAM file. People checking by hand can easily see “the job disappeared from the queue” and wrongly assume success.
- **It prevents human error:** you can’t start a step too early, forget to start it, or start it with the wrong input.
- **It is efficient:** all jobs are submitted at once, so they are already in the queue, and the pipeline doesn’t wait for you to resubmit each stage. Pending dependent jobs use no CPUs or memory until they start.

- **It is reproducible and documented:** the dependency chain in the script records the order of the workflow, so others can rerun the whole pipeline with the same logic.
- **It scales:** for hundreds of samples, manual checking is impractical, while a dependency like afterok on an array handles them all.
- **It fails safely:** if an upstream job fails, the dependent job never runs (DependencyNeverSatisfied), which saves compute time and avoids wrong results.

Part J — Troubleshooting

Scenario 1: OUT_OF_MEMORY at 16 GB requested, about 15.9 GB peak

What probably happened

The job used essentially all of the 16 GB it asked for and then crossed the limit. The 15.9 GB peak is what was recorded at the last sampling point before SLURM killed the job, so the true demand was probably higher still. Memory is usually checked at intervals, and the program may have tried to allocate more between checks. A job that was killed at the limit has not finished, so 15.9 GB is a **floor** for its real need, not the real peak.

Common causes in bioinformatics are an input larger than expected, a tool whose memory grows with the data (sorting, assembly, loading a large index), or too many threads, since each thread can add its own buffers.

Scenario 2: TIMEOUT at 02:00:00, program still processing

What to investigate before resubmitting

Simply doubling the time is a guess. First find out why the job needed more than two hours.

- **How far did it get?** Check the log for progress (reads processed, chromosomes completed, percent done). If it was 95% done, a slightly longer limit fixes it. If it was 10% done, the estimate was badly wrong and the job needs rethinking.
- **Was it making progress or stuck?** An endless loop, a hung process or a program waiting for input looks the same as a slow job. Check whether the log or output file was still growing.
- **Was the input larger than expected?** A bigger FASTQ file, more samples or a larger reference than in earlier runs would explain it.
- **Were the resources adequate?** Check that the program used all the CPUs it was given (--threads "$SLURM_CPUS_PER_TASK"). If it ran on 1 thread while 8 were allocated, the fix is to use the allocated CPUs, not to extend the time.

- **Was there a bottleneck?** Slow reading or writing on shared storage, or memory pressure, can stretch the runtime.
- **Does the tool support checkpoints or restarts?** If so, it can resume instead of starting again from zero, which saves time.
- **How long did similar earlier jobs take?** Test on a small subset to estimate the runtime per sample before choosing a limit.

Scenario 3: 64 CPUs, 256 GB, 48 hours requested; actual use was low CPU, 12 GB peak, 35 minutes

Why this is poor HPC practice

Compare what was requested with what was used:

Resource	Requested	Actually used	Usage
CPUs	64	very low utilization (probably only a few busy cores)	very small fraction
RAM	256 GB	12 GB	12 / 256 ≈ 4.7%
Time	48 hours (2,880 min)	35 min	35 / 2,880 ≈ 1.2%

The problems this causes:

- **Wasted, blocked resources:** the cluster reserves the full request for the whole run. For 35 minutes, 64 CPUs and 256 GB were unavailable to other users even though almost none of it was being used.
- **Longer queue wait:** a big request is hard for the scheduler to fit, so the job probably waited far longer than a small one would.
- **Hurts other users and yourself:** the shared cluster runs less efficiently, and fair-share systems charge for what was **allocated**, so your future jobs may get lower priority.
- **Poor planning:** it shows the request was a guess, not based on measurement. A very long time limit also makes it harder for the scheduler to backfill the job into gaps.
- **Likely wrong thread use:** very low CPU utilization suggests the program is single-threaded or not using the allocated cores, so the extra CPUs gave no speedup.

More appropriate request

Based on the measured usage:

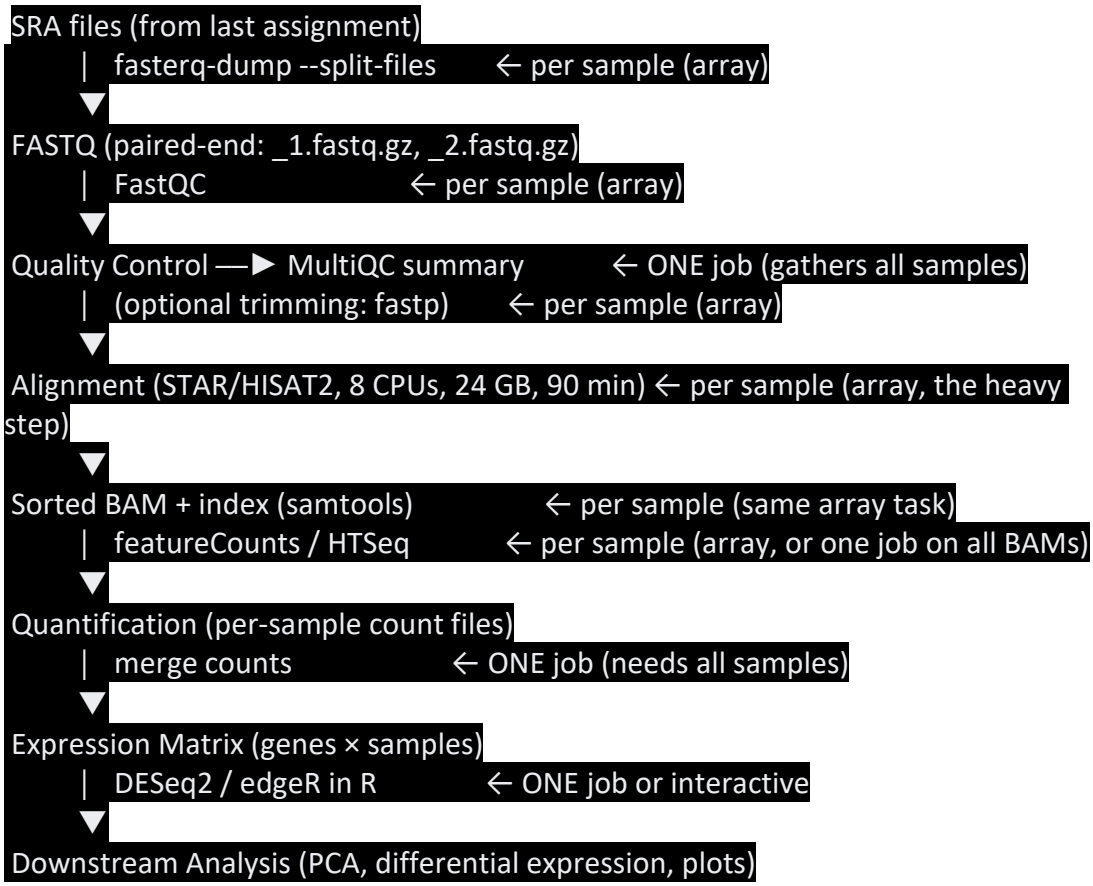
bash
```
#SBATCH --cpus-per-task=4#SBATCH --mem=16G#SBATCH --time=01:00:00
```

Reasoning

- **Memory, 16 GB:** the peak was 12 GB, so 16 GB leaves about a third of headroom (12 × 1.33 ≈ 16). This protects against OUT_OF_MEMORY if the data is slightly larger, without reserving 256 GB.
- **Time, 1 hour:** the job ran for 35 minutes, so 1 hour is about 1.7 times the runtime. That covers normal variation and avoids a timeout, while staying far below 48 hours.
- **CPUs, 4:** utilization was very low, so most of the 64 cores did nothing. First check whether the program actually supports multithreading. If it is single-threaded, request **1 CPU**. If it is multithreaded, 4 is a reasonable start, and the program should use --threads "$SLURM_CPUS_PER_TASK". Then check the utilization and increase the count only if the cores were busy.

Part K — Final Bioinformatics Workflow Design

1. Workflow diagram and parallelization



Stage	Parallel by sample?	Why
SRA → FASTQ	**Yes**	Each accession is independent
FastQC	**Yes**	Each file is analysed alone
MultiQC	No	Needs the reports from all samples

Stage	Parallel by sample?	Why
Trimming (optional)	**Yes**	Per sample
Alignment	**Yes**	Heaviest step, best use of an array
BAM sorting and indexing	**Yes**	Per sample, done inside the alignment task
Quantification	**Yes**	Per-sample counts (or one job over all BAMs)
Merge into expression matrix	No	Combines all samples
Downstream analysis	No	Compares samples with each other

So the **per-sample** stages go in job arrays, and the **all-samples** stages are single jobs that run only after the arrays succeed.

2. Resource strategy

Use a job array? Yes. Every sample needs the same steps with the same resources, and the samples are independent. One script handles all of them, and a failed sample can be rerun alone.

Item	Choice	Reasoning
CPUs per task	**8**	The stated requirement. Pass it to the aligner with --runThreadN "$SLURM_CPUS_PER_TASK"
RAM per task	**28 GB**	The need is about 24 GB, so this adds roughly 15% headroom against OUT_OF_MEMORY
Wall time	**02:15:00**	90 min × 1.5, covering variation in file size without a long, hard-to-schedule limit
Array size	1-N	One task per sample (1-10 for 10 samples)
Max concurrent tasks	%10	Keeps the footprint predictable and the shared file system healthy

Maximum simultaneous request with %10:

CPUs = 8 × 10 = 80 CPUs
RAM = 28 GB × 10 = 280 GB

For 100 samples this means 10 waves, so the total time is about 10 × 90 min = 15 hours instead of 90 min × 100. Adjust % to the cluster's limits. A smaller one (for example %5) gives 40 CPUs and 140 GB.

Lighter steps get smaller requests. FastQC, for example, needs about 2 CPUs and 4 GB for 20 minutes, and the merge and DESeq2 jobs need about 1-2 CPUs and 8 GB.

3. Directory structure

```
~/rnaseq_project/
 ├── config/
|    ├── samples.txt          # one sample/SRR accession per line
|    └── environment.yml      # exported conda environment
 ├── scripts/
|    ├── 01_fastq.sh   02_qc.sh   03_align.sh
|    ├── 04_quant.sh   05_merge.sh   06_downstream.R
|    └── submit_pipeline.sh    # chains jobs with dependencies
 ├── data/
|    ├── sra/                # downloaded SRA files
|    ├── fastq/              # sample_1.fastq.gz, sample_2.fastq.gz
|    └── reference/          # genome.fa, annotation.gtf, STAR index
 ├── results/
|    ├── qc/                 # FastQC per sample, multiqc_report.html
|    ├── alignment/<sample>/    # <sample>.sorted.bam, .bai, STAR logs
|    ├── counts/             # <sample>.counts.txt
|    ├── matrix/             # expression_matrix.tsv
|    └── downstream/         # PCA, DE tables, plots
└── logs/
    ├── fastq/  qc/  align/  quant/  merge/
```

Logs are organised **by stage**, with the array and task IDs in the name, for example logs/align/align_%A_%a.out. This makes it easy to find one sample's log, because the task number maps to a line in samples.txt.

4. Software environment

Use a dedicated **conda environment** with pinned versions, so the tools do not depend on whatever is installed on the server:

bash

```
conda create -n rnaseq -c conda-forge -c bioconda \  sra-tools=3.0.10 fastqc=0.12.1 multiqc=1.21 fastp=0.23.4 \  star=2.7.11b samtools=1.19 subread=2.0.6conda activate rnaseqconda env export > config/environment.yml
```

Each script starts with source ~/miniconda3/etc/profile.d/conda.sh and conda activate rnaseq, so the job gets the same tools every time. Tool versions are printed into the logs.

5. Main alignment array script

bash

```
#!/bin/bash#SBATCH --job-name=align#SBATCH --array=1-10%10#SBATCH --cpus-per-task=8#SBATCH --mem=28G#SBATCH --time=02:15:00#SBATCH --output=logs/align/align_%A_%a.out#SBATCH --error=logs/align/align_%A_%a.errset
```

```
-euo pipefailsource ~/miniconda3/etc/profile.d/conda.shconda activate
rnaseqSAMPLE=$(sed -n "${SLURM_ARRAY_TASK_ID}p"
config/samples.txt)OUT=results/alignment/${SAMPLE}mkdir -p "$OUT"echo "Job:
$SLURM_ARRAY_JOB_ID  Task: $SLURM_ARRAY_TASK_ID  Sample: $SAMPLE"echo
"CPUs: $SLURM_CPUS_PER_TASK  Node: $(hostname)  Start: $(date)"STAR --
versionSTAR --runThreadN "$SLURM_CPUS_PER_TASK" \     --genomeDir
data/reference/star_index \     --readFilesIn data/fastq/${SAMPLE}_1.fastq.gz
data/fastq/${SAMPLE}_2.fastq.gz \     --readFilesCommand zcat \     --outSAMtype
BAM SortedByCoordinate \     --outFileNamePrefix "$OUT/${SAMPLE}."samtools
index -@ "$SLURM_CPUS_PER_TASK"
"$OUT/${SAMPLE}.Aligned.sortedByCoord.out.bam"touch "$OUT/DONE"        #
marker file: only written if every step above succeededecho "End: $(date)"
```

set -euo pipefail makes the task stop with a non-zero exit code on the first error, so SLURM reports it as FAILED rather than COMPLETED. The thread count comes from $SLURM_CPUS_PER_TASK, as in Part F.

6. Job chain with dependencies

bash

```
FQ=$(sbatch --parsable --array=1-10%10 scripts/01_fastq.sh)QC=$(sbatch --parsable
--dependency=afterok:$FQ --array=1-10%10 scripts/02_qc.sh)AL=$(sbatch --parsable
--dependency=afterok:$FQ scripts/03_align.sh)QU=$(sbatch --parsable --
dependency=afterok:$AL scripts/04_quant.sh)MG=$(sbatch --parsable --
dependency=afterok:$QU scripts/05_merge.sh)sbatch --dependency=afterok:$MG --
wrap="Rscript scripts/06_downstream.R"
```

QC and alignment both depend only on the FASTQ step, so they run side by side. Merging and downstream analysis start only after **every** sample has succeeded.

7. Monitoring failures

- **While running:** squeue -u $USER and scontrol show job <id>.
- **Logs:** ls -l logs/align/ and cat logs/align/*.err. Find non-empty error files with find logs -name "*.err" -size +0.
- **Accounting (where enabled):** sacct -j <arrayID> --format=JobID,State,Elapsed,MaxRSS,ExitCode. Look for FAILED, TIMEOUT and OUT_OF_MEMORY. Your server has accounting disabled, so the logs and the marker files below become the main evidence there.
- **Marker files:** find samples whose alignment did not finish:

bash

```
 for s in $(cat config/samples.txt); do    [ -f results/alignment/$s/DONE ] || echo
"MISSING: $s"  done
```

- **Email alerts:** #SBATCH --mail-type=FAIL --mail-user=you@example.com (if the cluster supports mail).

- **Rerun only failures:** if tasks 3 and 7 failed, run sbatch --array=3,7 scripts/03_align.sh. Fix the cause first: more memory for OOM, more time for TIMEOUT.
- **Tune from measurements:** after the first run, check MaxRSS and Elapsed and adjust the request to about 20-30% above the peak.

8. Reproducibility

- **Pinned software:** environment.yml with exact versions, and tool versions printed in each log.
- **Fixed references:** record the genome and annotation version (for example GRCh38, Ensembl release), their download source, and their md5sum.
- **Recorded inputs:** SRA accessions in samples.txt, and checksums of the FASTQ files.
- **Scripts under version control:** keep the scripts/ and config/ folders in git, and note the commit used for the run.
- **Resources written in the script:** all #SBATCH lines are in the file, not typed ad hoc.
- **Logs kept per task:** each run's .out and .err files are archived, with the array ID in the name.
- **One command to rerun:** submit_pipeline.sh reproduces the full chain.
- **Fixed random seeds** in downstream analysis (set.seed() in R), plus sessionInfo() saved with the results.
- **A README** describing the folder layout, the order of the scripts and the run command.